

National University of Computer and Emerging Sciences

FAST-NUCES, Karachi Campus

Real-Time Messaging Application

A Flutter-Based Chat Application with SQL Backend

Project Report

CS — Database Systems

Student	Roll Number
	24K-1033
	24K-0851
	24K-0956

May 10, 2026

Contents

1	Introduction	2
2	System Overview	2
3	Database Design	2
3.1	Entity-Relationship Overview	2
3.2	Schema Definition	3
3.3	Table Descriptions	3
4	Application Architecture	3
4.1	Component Diagram	3
4.2	Key Modules	4
5	Key Features	4
6	Challenges and Solutions	5
7	Conclusion	5

Introduction

This report presents the design and implementation of a real-time messaging application developed using the Flutter framework with a PostgreSQL relational database backend managed through Supabase. The application allows users to register, authenticate, and exchange messages in real time with other registered users.

The primary objective of this project was to demonstrate the integration of a structured SQL database into a modern mobile application, replacing the conventional NoSQL approach typically associated with Flutter-based chat applications.

System Overview

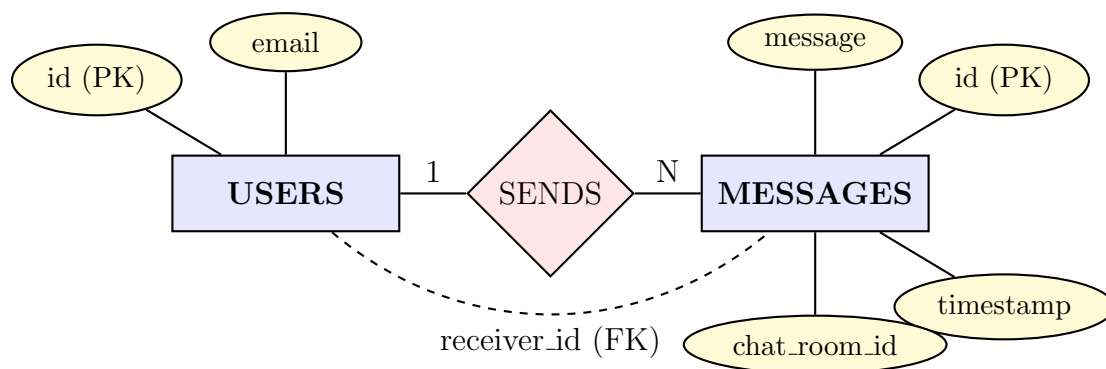
The application is built on a client-server architecture. The Flutter frontend communicates with a Supabase backend, which provides both authentication services and a hosted PostgreSQL database. The key components of the system are:

- **Flutter Frontend:** Cross-platform UI framework handling all user interaction.
- **Supabase Auth:** Manages user registration and session handling.
- **PostgreSQL (via Supabase):** Stores user profiles and chat messages.
- **Realtime Streams:** Supabase stream subscriptions provide live message updates.

Database Design

Entity-Relationship Overview

The database consists of two primary tables: `users` and `messages`. A user can send many messages, and each message is associated with both a sender and a receiver.



Schema Definition

The SQL schema used to create the database tables in Supabase is as follows:

```
1 CREATE TABLE users (  
2     id    UUID PRIMARY KEY,  
3     email TEXT NOT NULL  
4 );  
5  
6 CREATE TABLE messages (  
7     id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
8     sender_id     UUID REFERENCES users(id),  
9     receiver_id   UUID REFERENCES users(id),  
10    chat_room_id  TEXT NOT NULL,  
11    message       TEXT NOT NULL,  
12    timestamp     TIMESTAMPTZ DEFAULT now()  
13 );
```

Listing 1: Database Schema

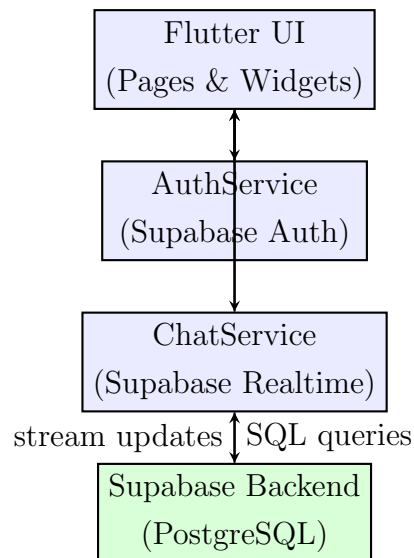
Table Descriptions

Table	Column	Description
users	id	UUID from Supabase Auth, primary key
	email	Registered email of the user
messages	id	Auto-generated UUID for each message
	sender_id	FK referencing users.id
	receiver_id	FK referencing users.id
	chat_room_id	Concatenation of sorted user IDs
	message	Text content of the message
	timestamp	Time of message creation

Application Architecture

Component Diagram

The following diagram illustrates the high-level flow of data between the Flutter application layers and the Supabase backend.



Key Modules

- **AuthGate:** Listens to Supabase auth state changes and routes the user to either the home screen or the login/register screen.
- **AuthService:** Wraps Supabase authentication calls for sign-up, sign-in, and sign-out. On registration, a corresponding record is inserted into the `users` table.
- **ChatServices:** Handles sending messages to the `messages` table and streaming incoming messages filtered by `chat_room_id`.
- **ThemeProvider:** A `ChangeNotifier` that manages light/dark theme toggling across the application.

Key Features

- User registration and authentication via Supabase Auth.
- Real-time message delivery using Supabase stream subscriptions.
- Persistent chat history stored in PostgreSQL.
- Dynamic chat room ID generation based on sorted user UUIDs ensuring unique bilateral conversations.
- Light and dark theme support.
- Cross-platform support (Android, Web).

Challenges and Solutions

Challenge	Solution
Migration from Firestore (NoSQL) to PostgreSQL (SQL)	Redesigned data model into normalized relational tables and replaced all Firebase SDK calls with Supabase equivalents.
Row Level Security blocking inserts	Disabled RLS on <code>users</code> and <code>messages</code> tables during development.
BuildContext across async gaps	Added <code>mounted</code> checks and stored context-derived objects before awaiting.
Supabase stream incompatibility with <code>QuerySnapshot</code>	Replaced <code>StreamBuilder<QuerySnapshot></code> with <code>StreamBuilder<List<Map<String, dynamic>>></code> .

Conclusion

This project successfully demonstrates the use of a relational SQL database (PostgreSQL via Supabase) as the backend for a real-time Flutter messaging application. The migration from Firebase Firestore to a structured SQL schema reinforces the practical applicability of database normalization, foreign key constraints, and SQL querying in a production-like mobile application context.